

Final Grade Calculation	1
Retake/ Re-examination / Grade Improvement in the 2025–2026 Academic Year	1
Project Topics	1
Requirements.....	1
Repository must include:.....	2
Useful links for finding scientific papers:.....	2
Academic Integrity	2
Reference Template:	2
T1 – Text-To-Sql.....	2
T2 – Relational vs. NoSQL	2
T3 – Schema design strategies and scaling solution in MongoDB.....	2
T4 – Caching and data acceleration with Redis	2
T5 – Graph databases	2
T6 – Semantic search engine	2
T7 – Comparisons and performance analysis	2
T8 – Study and presentation of a scientific article (with prior approval)	2
T9 – Custom topic proposed by the team (with prior approval)	2

Final Grade Calculation

The final grade will be determined as follows:

- **3 points – Team project**, submitted electronically **one week before the scheduled exam date** (project requirements are described on the following pages);
- **4 points – Project presentation during the exam session** (students may be asked questions about their work);
- **1 point by default**;
- **2 points – Laboratory activity and performance.**

To **pass the course**, a student must obtain a **minimum total of 5 points (without rounding)**.

Retake/ Re-examination / Grade Improvement in the 2025–2026 Academic Year

The grading rules are **identical to those announced above**.

The **points awarded for laboratory activity cannot be recovered or increased** after the end of the **first semester of the current academic year**.

Project Topics

Requirements

- Projects may be developed in **teams of up to 2 students**. Teamwork is preferred but not mandatory.
- Each team must choose **one topic** from the list below.
Each topic may be selected by **up to 3 teams**, except for **T8**, which may be chosen by more than 3 teams if the selected scientific articles differ, and **T9**, under the same condition.
- **Deadlines:**
 - Team and topic selection: **3 weeks, 30.10, 10:00 a.m.**
 - Progress status (lab presentation and upload to repository)
 - Final project submission: **TBA (a week before exam)** (upload to repository)
- For each missed deadline, **0.75 points** will be deducted from the final grade.
- All team members will receive the same grade (except in cases where a member has committed to completing certain tasks but failed to do so).
- The **team composition, chosen topic, repository link** (on GitHub / GitLab / BitBucket), and **team members' emails** must be submitted via the form: <https://forms.gle/uJCJsSgSgSvDsH9k8>

Repository must include:

- Functional **source code**
- A **README/wiki** containing:
 - **Technical documentation:** system architecture, data model, software/hardware configurations, VM usage (if any), tool versions, relevant code snippets, screenshots of code execution and tool outputs, tabular

comparisons of results/tools, interpretations, and **bibliographic references** (properly cited in-text).

- o **Diagrams** must be created using dedicated tools (e.g. <https://app.diagrams.net>, <https://www.lucidchart.com>, yEd, Microsoft Visio, etc.). **Hand-drawn or scanned images are not accepted.**
- A **presentation** (PowerPoint, Keynote, Canva, Remark.js + Markdown + KaTeX, etc.) summarizing the documentation (max. **10 slides**).
- **Video demonstrations** of code execution (e.g. YouTube, Microsoft Stream).
- Each team member must have **commit activity** in the repository.
- References may include: official tool documentation, scientific articles, and books.

Useful links for finding scientific papers:

- <https://scholar.google.com>
- <https://dblp.uni-trier.de>
- <https://dl.acm.org>
- <https://ieeexplore.ieee.org>
- <https://arxiv.org>
- <https://www.researchgate.net>
- <https://www.sci-hub.st>
- <http://libgen.gs>

Academic Integrity

The project must be **original** (see FMI's Ethics and Professionalism Regulations – item 11, page 3, on minor incidents).

This means the project **cannot include verbatim text** copied from other sources.

However, **consulting** and **citing** diverse sources of information is encouraged, including automatically generated ones.

Quotations (with quotation marks) are permitted when appropriate.

Documentation and code will be automatically checked with **Turnitin** for similarity.

Text copied directly from other sources will be disregarded, and the grade will be reduced accordingly.

You may use **AI tools** to generate **text, code, images, or videos**, or to transform text into music or video (e.g., Claude, ChatGPT, Microsoft Copilot, Gemini, rikigpt.com, jenni.ai, Microsoft Designer, DALL-E, InVideo), **as long as this is explicitly acknowledged**.

Reference Template:

- [1] Author Last name, First name, Online article title, URL, Last accessed: ...
- [2] Author Last name, First name, Scientific article title, Journal name, vol. X, no. Y, Year, Pages.
- [3] Author Last name, First name, Book title, Publisher, Year.
- [4] OpenAI, ChatGPT, <https://chatgpt.com/>, Date generated: March 1, 2025.
- [5] Copilot, <https://copilot.microsoft.com>, Date generated: March 1, 2025.
- [6] Bing, <https://www.bing.com/create>, Date generated: March 1, 2025.

T1 – Text-To-Sql

Build a multi-step chain (using LangChain or other libraries at your choice) that:

- Accepts an NLP question,
- Generates a SQL query,
- Executes it,
- Summarizes the result in natural language.

You should:

Handle ambiguous or vague NLP input

- use prompt engineering or schema-aware techniques to guide the LLM
- include schema context in the prompt;

Compare SQL query generation across different LLMs

- evaluate differences in query accuracy, syntax, and logic;

Handle errors and optimize using feedback

- add mechanisms to detect and correct invalid SQL
- implement user feedback loop (e.g., thumbs up/down, corrections)
- optionally explore *few-shot* or *chain-of-thought prompting*

Test the solution on known benchmarks

- Use Spider or Bird to measure *Exact match accuracy* and *Execution accuracy*.

Add UI (optional): simple web or CLI interface.

T2 – Relational vs. NoSQL

Compare relational and NoSQL database systems from architectural, consistency, and scalability perspectives.

You should:

- Design an experiment comparing at least two database types (e.g., PostgreSQL vs. MongoDB, or MySQL vs. Cassandra).
 - Model the same dataset (e.g., e-commerce, social network, IoT telemetry) in both systems.
 - Highlight how data modeling differs in relational vs. NoSQL.
- Demonstrate CAP vs. ACID trade-offs:
 - Identify how each system handles consistency, availability, and partition tolerance.
 - Simulate a network partition or node failure and describe the behavior.
- Evaluate query complexity and performance:
 - Compare latency for simple and aggregate queries.
 - Use metrics like response time, throughput, availability under stress.
- Document architecture decisions:
 - Justify the selected databases and configurations.
 - Include ER and NoSQL schema diagrams.
- (Optional) Add visualization:
 - Dashboard comparing metrics or visualizing CAP trade-offs.

Deliverables: Technical report + scripts + dataset + screenshots/metrics.

T3 – Schema design strategies and scaling solution in MongoDB

Build a project (using MongoDB and optional driver libraries like PyMongo, Mongoose, or others) that:

- Designs and organizes collections for a chosen domain (e.g., e-commerce, IoT devices, resource/content management),
- Implements CRUD operations (Create, Read, Update, Delete),
- Applies and evaluates scaling strategies (indexing, replication, sharding),
- Demonstrates trade-offs in schema design and scaling approaches.

You should:

Demonstrate Data Modeling and Collection Organization

- define entities and relationships
- compare embedding vs referencing strategies and justify schema choice
- implement at least 2 different schema models and discuss pros/cons;

Implement CRUD Operations

- implement Create, Read, Update, Delete with realistic queries
- include aggregation pipelines for more complex queries
- (Optional) Expose CRUD functionality via a simple API (Python Flask/FastAPI or Node.js);

Apply Scaling & Optimization strategies

- apply indexes and evaluate query performance (before/after)
- set up replication (replica set) and test read/write failover
- simulate sharding with a large dataset, test query performance, and analyze trade-offs
- compare vertical vs horizontal scaling scenarios and explain when each is preferable;

Handle errors and evaluate

- Handle invalid queries
- Document common pitfalls in schema design and CRUD operations
- Reflect on trade-offs between flexibility, performance, and consistency in NoSQL design

- Measure query performance with/without indexing
- Compare schema strategies (embedded vs referenced documents)
- Document performance changes when scaling (replication, sharding)
- Discuss when MongoDB is a better fit than SQL for your chosen dataset.

Add UI (optional): simple web or CLI interface interacting with collections.

T4 – Caching and data acceleration with Redis

Build a project (using Redis or compatible in-memory data stores) that:

- Implements a caching layer to accelerate queries and responses from databases or APIs.
- Demonstrates cache strategies such as read-through, write-through, and cache-aside.
- Evaluates cache hit/miss ratios, latency improvements, and scalability under load.
- Integrate Caching into an existing system.

You should:

Design and Integrate a Caching Architecture

- Identify a system or dataset where caching improves performance (e.g., frequent queries in MongoDB, repeated semantic searches, or API results).
- Design the cache hierarchy and interaction flow between the application layer, cache, and persistent database.
- Implement one or more caching strategies:
 - o Cache-aside: Application checks Redis before querying the database.
 - o Read-through: Cache automatically populates on read miss.
 - o Write-through / Write-behind: Data written to Redis and persisted asynchronously to the main database.
- Document architectural diagrams showing data flow.

Implement Core Features

- Use Redis data structures such as:
 - Strings (simple key-value pairs)
 - Hashes (object-style caching)
 - Lists/Sets (for query result caching)
 - Sorted Sets (for ranking or scoring systems)
 - Stream (event-driven caching, real-time updates, or log processing)
 - Geospatial (for proximity searches or region-based caching)
- Implement TTL (Time-To-Live) and LRU (Least Recently Used) eviction policies to manage cache memory.
- Simulate cache invalidation policies for updates, deletions, and stale data handling.

Add UI (optional): Build a simple dashboard that displays:

- Latency comparisons between cached and non-cached queries.
- Visualize metrics with Grafana or Python dashboards: cache hit ratio, average latency reduction, eviction rates.

T5 – Graph databases

Build and query a graph database that reveals non-trivial relationships between entities.

You should:

- Model a graph dataset (can be real-world or synthetic) — e.g., social connections, fraud detection, academic citations, transport routes, or movie recommendations.
 - Define nodes, relationships, and key properties.
- Implement queries using Cypher (Neo4j) or AQL (ArangoDB):
 - Graph traversal (e.g., shortest path, friends-of-friends).
 - Pattern matching (e.g., users with similar interests, co-authorship detection).
- Evaluate efficiency and expressiveness:
 - Compare query readability and performance against an equivalent relational model.
 - Use metrics: query time, number of returned nodes/edges.

- Integrate with a Python or JavaScript API:
 - Simple interface to visualize graph traversal or to input a node and get results dynamically.
- (Optional) Apply an AI or analytics layer:
 - Use graph embeddings or community detection to enhance recommendations.

Deliverables: Database dump + scripts + report explaining modeling and results.

T6 – Semantic search engine

Build a project (using a vector database or search framework such as FAISS, Chroma, Weaviate, Qdrant, or Elasticsearch with embeddings) that:

- Indexes a corpus of text documents (articles, reports, FAQs, transcripts, etc.) using vector embeddings.
- Enables semantic search — retrieving documents based on meaning, not just keywords.
- Integrates an LLM (via LangChain, LlamaIndex, or similar) to summarize or answer queries based on retrieved content.
- Evaluates retrieval quality, accuracy, and latency across different search and ranking strategies:

You should:

Implement Data Ingestion and Embedding:

- Choose or create a dataset (e.g., Wikipedia articles, product descriptions, course notes).
- Generate vector embeddings using models such as OpenAI, SentenceTransformers.
- Store embeddings and metadata in a vector-capable backend (Chroma, FAISS, Weaviate, Qdrant, Elasticsearch, etc.).
- Organize your data schema with document chunks, titles, sources, and IDs.

Build and Compare Search Strategies

- Implement different retrieval approaches:
 - Keyword-based search (BM25, TF-IDF, or Elasticsearch).

- o Semantic search (pure embedding similarity).
 - o Hybrid search (combining keyword + vector similarity).
- Compare different retrieval approaches performance and relevance on a common query set.
- Integrate Approximate Nearest Neighbor (ANN) Search:
 - o Implement and compare exact vs approximate nearest neighbor search using vector indexing methods such as HNSW, IVF, or PQ (depending on the database).
 - o Evaluate latency improvements and recall trade-offs between exact and approximate retrieval.
 - o Analyze how ANN parameters impact retrieval quality and speed.
- Include Range Search Analysis:
 - o Implement range search queries to retrieve all vectors within a fixed similarity or distance threshold (e.g., cosine similarity > 0.8).
 - o Compare range search with top-k retrieval to study trade-offs:
- (Optional) Integrate LLM for Query Understanding and Answer Generation: Implement reranking, provide relevance scores, produce a concise, contextualized answer.

Add UI (optional): simple web or CLI interface that allows users to:

- Input natural language queries.
- View top retrieved documents with similarity scores.
- Display LLM-generated summaries or answers.
- Visualize search performance metrics.

T7 – Comparisons and performance analysis

Conduct a mini-benchmark study comparing different database paradigms (document, column-store, and graph).

You should:

- Select or generate a dataset usable across all three systems (e.g., user interactions, product catalog, or transactions).

- o Store equivalent information in MongoDB, Cassandra, and Neo4j.
 - Design representative workloads:
 - o CRUD operations (Create/Read/Update/Delete).
 - o Analytical or aggregation queries.
 - o Graph traversal or multi-table joins.
 - Measure and compare:
 - o Insertion speed, query latency, scalability (with increasing data volume), and resource usage (CPU/memory).
 - o Document the testing environment and methodology (e.g., batch size, replication factor).
 - Analyze trade-offs:
 - o Highlight when each DB type excels or fails.
 - o Interpret results based on data model and consistency mechanism.
 - (Optional) Automate benchmark execution with Python scripts or JMeter.
- Deliverables: Benchmark scripts + report with graphs/tables summarizing results + reproducibility notes.

T8 – Study and presentation of a scientific article (with prior approval)

- Choose a **scientific paper published within the last 4 years** on a topic relevant to the course (NoSQL databases, graph/document/columnar models, applied AI, etc.). Suggested sources: <https://scholar.google.com> , <https://dblp.uni-trier.de> , <https://dl.acm.org> , <https://ieeexplore.ieee.org>
- Study and understand the key concepts, methods, and software tools discussed in the paper.
- Discuss your choice with the instructor for **approval**. *(Fill in the last section of the form — “Observations/comments/description T8, T9.”)*
- Write a **report** that synthesizes the article’s contributions and explains them in accessible language.
- Develop your own **demo** illustrating the concepts or methods from the paper (without reproducing the original example exactly).
- Include the paper in your **project bibliography** and cite it properly in the documentation.

T9 – Custom topic proposed by the team (with prior approval)

- Teams may propose **original or applied topics** within the course domain: NoSQL databases, graph/document/columnar models, AI integration, or modern query and analysis tools.
- Topics involving **emerging technologies** (e.g., vector databases, LLMs, AutoML, new query engines) are encouraged.
- The topic must be **discussed with and approved by the instructor** before implementation.

(Fill in the last section of the form — “Observations/comments/description T8, T9.”)